
Exercice 1 : Décorateur de mise en cache avec expiration (Niveau Moyen)

Énoncé :

Crée un décorateur `@memoize_avec_ttl(ttl_secondes: int)` qui met en cache le résultat d'une fonction pendant une durée déterminée (`ttl_secondes`).

1. Si la fonction est appelée avec les mêmes arguments dans l'intervalle de temps imparti, le résultat stocké dans le cache doit être renvoyé sans réexécuter la fonction.
 2. Si le délai est dépassé, la fonction doit être réexécutée et le cache mis à jour.
 3. N'oublie pas d'utiliser `functools.wraps`.
-

Exercice 2 : Gestionnaire de contexte & Générateur pour la lecture de fichiers volumineux (Niveau Intermédiaire)

Énoncé :

Lors du traitement de gros fichiers journaux (*logs*), charger l'intégralité du fichier en mémoire est inefficace.

1. Crée un gestionnaire de contexte `@contextmanager` nommé `ouvrir_journal(chemin: str)` qui ouvre un fichier en lecture et s'assure de sa fermeture à la sortie du bloc.
 2. Écris une fonction génératrice `filtrer_logs(fichier, niveau: str)` qui prend le fichier ouvert et lit les lignes une par une (`yield`), en ne renvoyant que les lignes contenant le niveau spécifié (ex: `"ERROR"`, `"WARNING"`).
-

Exercice 3 : Collecteur de données asynchrone avec `asyncio` (Niveau Avancé)

Énoncé :

Simule un système qui interroge plusieurs API en parallèle.

1. Écris une coroutine `fetch_api(service_nom: str, delai: float) -> dict` qui attend `delai` secondes (avec `asyncio.sleep`) pour simuler un appel réseau, puis renvoie `{"service": service_nom, "status": 200}`.
 2. Écris la coroutine principale `main()` qui lance en parallèle les appels suivants :
 - `"Service Utilisateurs"` (délai : 1.5s)
 - `"Service Paiements"` (délai : 0.8s)
 - `"Service Notifications"` (délai : 2.0s)
 3. Utilise `asyncio.gather` pour exécuter ces tâches de manière concurrente et affiche la durée totale d'exécution (qui doit être proche du délai le plus long, soit ~2 secondes, et non de la somme des délais).
-

Solutions et Corrections

Correction Exercice 1

```
import functools
import time

def memoize_avec_ttl(ttl_secondes: int):

    def decorateur(func):
        cache = {} # Structure : {args_key: (resultat, timestamp)}

        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            # Clé unique pour le cache basée sur les arguments
            cle = (args, tuple(sorted(kwargs.items())))
            maintenant = time.time()

            if cle in cache:
                resultat, timestamp = cache[cle]
                if maintenant - timestamp < ttl_secondes:
                    print(f"[CACHE HIT] Valeur récupérée depuis le cache
pour {func.__name__}")
                    return resultat

                print(f"[CACHE MISS] Exécution de la fonction
{func.__name__}...")
                resultat = func(*args, **kwargs)
                cache[cle] = (resultat, maintenant)
                return resultat

            return wrapper

        return decorateur

# Test
@memoize_avec_ttl(ttl_secondes=2)
def calcul_couteux(x: int) -> int:
    return x * 2

print(calcul_couteux(10)) # Exécution
print(calcul_couteux(10)) # Récupéré depuis le cache

time.sleep(2.1) # Attente de l'expiration du TTL
print(calcul_couteux(10)) # Réexécution (cache expiré)
```

Correction Exercice 2

```
from contextlib import contextmanager
import os

@contextmanager
def ouvrir_journal(chemin: str):
    fichier = open(chemin, "r", encoding="utf-8")
    try:
        print(f"Ouverture du fichier log : {chemin}")
        yield fichier
    finally:
        print("Fermeture sécurisée du fichier log.")
        fichier.close()

def filtrer_logs(fichier_iter, niveau: str):
    """Générateur qui filtre les lignes à la volée (Lazy Evaluation)."""
    niveau_str = f"[{niveau.upper()}]"
    for ligne in fichier_iter:
        if niveau_str in ligne:
            yield ligne.strip()

# Création d'un fichier de test temporaire
fichier_demo = "app.log"
with open(fichier_demo, "w", encoding="utf-8") as f:
    f.write("[INFO] Démarrage application\n")
    f.write("[ERROR] Connexion DB échouée\n")
    f.write("[WARNING] Mémoire élevée\n")
    f.write("[ERROR] Timeout API\n")

# Test de notre pipeline de lecture
with ouvrir_journal(fichier_demo) as f:
    logs_erreur = filtrer_logs(f, "ERROR")
    for log in logs_erreur:
        print(f"Trouvé : {log}")

# Nettoyage du fichier de test
if os.path.exists(fichier_demo):
    os.remove(fichier_demo)
```

Correction Exercise 3

```
import asyncio
import time

async def fetch_api(service_nom: str, delai: float) -> dict:
```

```
    print(f"Démarrage de la requête vers {service_nom} (attente  
{delai}s)...")  
    await asyncio.sleep(delai) # Non-bloquant pour la boucle d'événements  
    print(f"Réponse reçue de {service_nom}")  
    return {"service": service_nom, "status": 200}  
  
async def main():  
    debut = time.perf_counter()  
  
    # Lancement parallèle avec asyncio.gather  
    resultats = await asyncio.gather(  
        fetch_api("Service Utilisateurs", 1.5),  
        fetch_api("Service Paiements", 0.8),  
        fetch_api("Service Notifications", 2.0),  
    )  
  
    fin = time.perf_counter()  
  
    print("\n--- Résultats ---")  
    for res in resultats:  
        print(res)  
  
    print(f"\nTemps total d'exécution : {fin - debut:.2f}s")  
  
# Exécution de la boucle d'événements  
asyncio.run(main())
```

Résultat attendu en console : Le temps total doit être d'environ **2.00s** (durée du service le plus lent), prouvant l'exécution asynchrone non-bloquante.
